claude-windows / ac48091f-a4f4-4268-b968-75205d9a50ff

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\ac48091f-a4f4-4268-b968-75205d9a50ff\subagents\workflows\wf_b8ec0894-3cb\agent-ab7431b7cf16c719b.jsonl`
- SHA-256: `ac8441d4b16313b7bcc2044ebdf9b56e22cf8a6791a4aa39d42862996b81c1b9`
- Source modified: `2026-06-21T19:13:37+00:00`
- Imported at: `2026-07-08T15:59:55+00:00`
- project: `wf_b8ec0894-3cb`
- session_id: `ac48091f-a4f4-4268-b968-75205d9a50ff`

Transcript

1. user (2026-06-21T19:12:53.040Z)

```
You are verifying ONE claim in khelsutra's deploy runbook (deploy/DEPLOY_ALPHA.md) against the
REAL code the owner will deploy.

CRITICAL repo rules:
- Engine repo is at /c/Users/avidu/Projects/badminton-highlight-indexer. Its working tree is
checked out on a STALE concurrent-session branch (12 commits behind master). DO NOT read the
working tree for engine claims and DO NOT checkout/commit/modify anything there ? it is a shared
checkout owned by another session.
- To read what the owner actually deploys, read ENGINE origin/master ONLY, via:
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer grep -n "PATTERN" origin/master
-- 'PATHGLOB'      (search master's tree)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer show origin/master:PATH
(read a master file)
    git -C /c/Users/avidu/Projects/badminton-highlight-indexer log origin/master --oneline -i
--grep="..."        (history)
  (I already ran 'git fetch' so origin/master is current. Never fetch/checkout.)
- khelsutra repo is at /c/Users/avidu/Projects/khelsutra, working tree is on clean master ? you
MAY use Read/Grep/Glob directly there.

Be adversarial and literal: confirm the claim is TRUE for origin/master, with file:line
evidence. If the doc says X but the code says Y, that is a 'discrepancy' ? capture it precisely.
If something the owner must DO (beyond the doc's text) is implied, put it in owner_action_note.
Verdict 'confirmed' only with concrete evidence; 'partial' if mostly true with a caveat;
'refuted' if wrong; 'unknown' if you genuinely could not determine it.

----
CLAIM (§2.2): the engine config has a `security` section with keys: edge_auth_enabled,
allowed_video_root, upload_dir, cf_team_domain, cf_access_aud ? all real, with the described
meaning.
On origin/master, locate the security config model (likely backend/config/). Use `git -C
/c/Users/avidu/Projects/badminton-highlight-indexer grep -n
"edge_auth_enabled\|allowed_video_root\|upload_dir\|cf_team_domain\|cf_access_aud" origin/master
-- 'backend/config/*'` then `git show` the model file. Confirm each of the 5 keys exists, the
section is named `security`, and report each key's type + default. Flag any key the doc names
that does NOT exist, or any default that contradicts the doc (e.g. if edge_auth_enabled defaults
true).
```

2. assistant (2026-06-21T19:12:57.833Z)

```
I'll verify this claim against engine origin/master. Let me start by reading the doc claim and
finding the security config model.
```

3. user (2026-06-21T19:12:59.532Z)

```
origin/master:backend/config/models.py:283:    allowed_video_root: Optional[str] = None
origin/master:backend/config/models.py:287:    # arrive with PR-3 identity scoping. ? In hosted
mode, set allowed_video_root to
origin/master:backend/config/models.py:288:    # contain upload_dir or /api/process will reject
the uploaded paths.
origin/master:backend/config/models.py:289:    upload_dir: str = "output/uploads"
origin/master:backend/config/models.py:297:    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT
is required/verified (the local single-user
origin/master:backend/config/models.py:303:    edge_auth_enabled: bool = False
origin/master:backend/config/models.py:304:    cf_team_domain: str = ""        # e.g.
https://<team>.cloudflareaccess.com (issuer + /certs JWKS)
origin/master:backend/config/models.py:305:    cf_access_aud: str = ""         # the CF Access
application AUD tag the token must carry
origin/master:backend/config/startup.py:19:behind an edge gate (the owner flips
``security.edge_auth_enabled`` on for the Cloudflare-Access
origin/master:backend/config/startup.py:22:rest of the posture at **boot**: the
``allowed_video_root`` path-jail is set (else an authenticated
origin/master:backend/config/startup.py:30:a strict **no-op** ? exactly the pre-M5 behaviour;
with ``edge_auth_enabled=False`` (today's default)
origin/master:backend/config/startup.py:91:    """Exposure-safety posture for an *exposed* HTTP
server. Gated on ``security.edge_auth_enabled``
origin/master:backend/config/startup.py:98:    if not sec or not getattr(sec,
"edge_auth_enabled", False):
origin/master:backend/config/startup.py:104:    # a tenant can point the engine at an ARBITRARY
local file (allowed_video_root unset = "any local
origin/master:backend/config/startup.py:106:    root = getattr(sec, "allowed_video_root", None)
origin/master:backend/config/startup.py:110:            "security.edge_auth_enabled is ON
(engine exposed) but security.allowed_video_root is "
origin/master:backend/config/startup.py:112:            "Set allowed_video_root to a jail
directory before exposing the engine.",
origin/master:backend/config/startup.py:118:    team = (getattr(sec, "cf_team_domain", "") or
"").strip()
origin/master:backend/config/startup.py:119:    aud = (getattr(sec, "cf_access_aud", "") or
"").strip()
origin/master:backend/config/startup.py:121:        missing = " + ".join(n for n, v in
(("cf_team_domain", team), ("cf_access_aud", aud)) if not v)
origin/master:backend/config/startup.py:124:            f"security.edge_auth_enabled is ON but
{missing} is not configured ? the Cf-Access JWT "
origin/master:backend/config/startup.py:125:            "cannot be verified (every request would
fail). Set security.cf_team_domain "
origin/master:backend/config/startup.py:126:            "(e.g.
https://<team>.cloudflareaccess.com) and security.cf_access_aud (the CF Access "
origin/master:backend/config/startup.py:136:            "security.edge_auth_enabled is ON but
PyJWT[crypto] is not importable ? Cf-Access "
origin/master:backend/config/startup.py:146:            "security.edge_auth_enabled is ON but
RALLY_ALLOWED_HOSTS is unset ? the Host-header guard "
origin/master:backend/config/startup.py:167:    DEFAULT-OFF: no profile ? no profile checks;
``edge_auth_enabled=False`` ? no exposure checks.
origin/master:backend/config/startup.py:174:    # runs with no profile set), gated on
edge_auth_enabled (default-OFF). Server-only.
```

4. user (2026-06-21T19:13:01.129Z)

```
19-```
20-
21:- **No inbound ports.** `cloudflared` dials out; the engine stays bound to `127.0.0.1:8000`.
The box's home/public IP is never exposed.
22:- **Auth at the edge.** Cloudflare Access gates both hostnames; the engine *also* verifies
the `Cf-Access` JWT in-app (defence in depth ? `security.edge_auth_enabled`), so a direct hit to
the tunnel can't spoof identity.
23:- **Cross-origin, on purpose.** `app.` (SPA) and `api.` (engine) are different origins ? the
SPA is built with `VITE_API_BASE=https://api.khelsutra.guru/api` (PR #64) and the engine locks
CORS to the SPA origin via `RALLY_CORS_ALLOW_ORIGINS` (no engine code change ? it's already env-
driven).
24-
```

25-> **Alternative (single-origin):** if you'd rather run **one box** that serves the SPA *and* the API behind one origin (Caddy reverse-proxy, no Pages, no cross-origin), see `LOCAL_APPLIANCE_ARCHITECTURE.md` D1. This runbook ships the **split** topology you chose (stable Pages-hosted page + a thin API tunnel).
--
47-Create / edit `config.json` on the box:
48-```jsonc
49-{
50: "security": {
51:    "edge_auth_enabled": true,   // turn the CF-Access JWT verify ON
52:    "allowed_video_root": "/srv/khelsutra/incoming", // the path-jail ? REQUIRED when exposed
53:    "upload_dir": "/srv/khelsutra/incoming/uploads", // MUST sit UNDER allowed_video_root (see §7)
54:    "cf_team_domain": "https://<your-team>.cloudflareaccess.com",
55:    "cf_access_aud": "<the CF Access application AUD>" // filled in §4
56- }
57-}
58-```
--
62-export RALLY_CORS_ALLOW_ORIGINS="https://app.khelsutra.guru"   # lock CORS to the SPA origin
63-```
64-
65:> **The engine refuses to start unsafe.** The boot posture (engine PR #294) **fails fast** if `edge_auth_enabled` is on but the `allowed_video_root` jail or `cf_team_domain`/`cf_access_aud` is missing, and **warns** if `PyJWT` isn't installed. So a half-configured exposure can't silently come up ? fix what it prints and restart.
66-
67-### 2.3 Run
68-```bash
--
93-1. **Zero Trust ? Access ? Applications ? Add ? Self-hosted.**
94-2. **Application domain:** add **both** `app.khelsutra.guru` **and** `api.khelsutra.guru` to the *same* application (one app ? one shared session cookie, so a tester signs in once and both the SPA and its API calls are authorized).
95-3. **Policy:** Action *Allow*, rule *Emails* ? your alpha testers' addresses (free ? 50 users ? fits 10?25 testers). Use one-time-PIN (email OTP) so testers need no Cloudflare account.
96:4. **Copy the application `AUD` tag** ? paste into the engine's `config.json` `security.cf_access_aud` (§2.2), and set `cf_team_domain` to `https://<your-team>.cloudflareaccess.com`. Restart the engine.
97-5. **CORS preflight ?:** the browser sends an unauthenticated `OPTIONS` preflight to `api.` for the cross-origin POSTs. In the Access application's **Settings ? enable ?Bypass options requests to CORS preflight?** (or add an explicit OPTIONS bypass), else the preflight is redirected to the login page and every API call fails. The engine's CORS middleware (`allow_credentials=false`, no cookies) then answers the preflight.
98-
99----
--
118-- `curl -i https://api.khelsutra.guru/api/health` **without** a CF session ? a **302 to the Access login** (not a 200). ? The gate is in front.
119-- Open `https://app.khelsutra.guru` in a browser ? CF Access login ? after sign-in, the SPA loads and `/api/health` returns 200 (same session cookie). ?
120-
121:**The shareable CUJ (what a tester does):** open `app.khelsutra.guru` ? sign in ? paste a `gs://?` video (or a path under `allowed_video_root`) ? **Process** ? watch progress ? pick rallies ? **Build** ? **download the reel**. All from the page.
122-
123----
124-
125-## 7. Honest caveats / deferred
126-
127:- **`upload_dir` must sit UNDER `allowed_video_root`.** If you enable browser upload, an upload landing outside the jail is rejected by `/api/process` (a functional break, not a hole). Keep `upload_dir` inside the jail (§2.2).
128:- **You MUST set `edge_auth_enabled=true` to expose.** The boot posture (and the in-app JWT

verify) only engage then. Do not put the engine behind the tunnel with edge auth off.
129-- **Cross-origin reel download:** the reel streams from `api.` with `Content-Disposition:
attachment`, so it downloads even cross-origin ? but verify the filename in your browser during
the CUJ (cross-origin `<a download>` filename hints are ignored; the header is the authority).
130-- **Upload size:** Cloudflare's free proxy caps request bodies at **100 MB** ? large matches
upload as ?90 MB chunks (the SPA chunked-upload path), or pre-stage to a bucket and ingest the
`gs://` URI (already supported).
131-- **Box offline = engine offline.** The Pages SPA stays up and shows a friendly "engine
offline" banner (`/api/health` ping); processing resumes when the box is back. Beta removes this
with a queue (`HOSTING_PLAN.md` §3).

5. assistant (2026-06-21T19:13:03.715Z)

All five keys are present. Let me read the full model file to confirm the section is named
`security`, and verify each key's type/default and the documented meaning.

6. user (2026-06-21T19:13:05.239Z)

```
    # Long-rally reel filter (seconds; 0 = OFF). Drop detected rally pairs whose duration
    # (end ? start) is below this from the highlight reel, BEFORE compiling. A cleaner
    # reel-selection knob than `min_shot_count`: shot_count is unlabeled / "Unknown" on the
    # fully-local no-AI path, whereas duration is derived from the rally's own start/end. The
    # local detector over-fires and its false positives are mostly SHORT (motion blips,
    # background-court fragments), so this keeps the reel to the eye-catching long rallies.
    # OFF by default ? the detector output is unchanged, only which rallies reach the reel.
    min_rally_duration: float = Field(default=0.0, ge=0.0)
    watermark: WatermarkConfig = Field(default_factory=WatermarkConfig)


class LoggingConfig(BaseModel):
    """Logging knobs for the run entrypoints (see backend/utils/logging_setup.py)."""
    # Third-party HTTP/RPC client loggers (httpx, httpcore, urllib3, google-genai,
    # google.auth, grpc) emit one INFO line per request ? dozens per Gemini run, which
    # buries our own [rally]/[compile] lines in run.log during manual QA. Default raises
    # them to WARNING; set true to restore full chatter for request-flow debugging.
    verbose_http: bool = False


class SecurityConfig(BaseModel):
    # When set, /api/process video_path must resolve under this directory (hosted/tenant
    # mode). Default None preserves the single-user local 'any local file' workflow.
    allowed_video_root: Optional[str] = None

    # --- Chunked upload (B2, PR-2) ---
    # Landing zone for browser uploads (assembled from parts). Per-user subdirectories
    # arrive with PR-3 identity scoping. ? In hosted mode, set allowed_video_root to
    # contain upload_dir or /api/process will reject the uploaded paths.
    upload_dir: str = "output/uploads"
    # Whole-file cap (default 4 GB) and per-part cap. Parts stay under ~100 MB because
    # free-tier edge proxies cap request bodies there (HOSTING_PLAN §2).
    max_upload_mb: int = 4096
    max_part_mb: int = 95
    allowed_upload_extensions: List[str] = ["mp4", "mov"]

    # --- M9 edge auth (Cloudflare Access, D9) ---
    # DEFAULT-OFF: edge_auth_enabled=False ? no JWT is required/verified (the local single-user
    # path is unchanged). When True, /api/process requires + verifies the `Cf-Access-Jwt-
Assertion`
    # header against the team JWKS (so a direct hit to the Cloud Run URL can't spoof the
identity
    # header) and tags the job with the verified user_id (owner_id). The team domain (e.g.
    # ``https://<team>.cloudflareaccess.com``) supplies the JWKS + issuer; the AUD is the CF
Access
    # application's audience tag. NO secrets here ? these are public identifiers.
    edge_auth_enabled: bool = False
```

```
    cf_team_domain: str = ""        # e.g. https://<team>.cloudflareaccess.com (issuer + /certs
JWKS)
    cf_access_aud: str = ""         # the CF Access application AUD tag the token must carry
    # (CORS allow-origins is set via the RALLY_CORS_ALLOW_ORIGINS env var ? read at import in
    # backend/main.py so importing the app does no filesystem I/O; default ["*"]. Credentials
stay
    # OFF ? Cf-Access is a header, not a cookie.)


class StorageConfig(BaseModel):
    """Pluggable storage backend (docs/DECOUPLED_COMPUTE/). Default ``local`` = today's
    behaviour (filesystem paths, byte-for-byte). ``s3`` covers AWS + Cloudflare R2 + DO Spaces
    + MinIO (via ``endpoint_url``); ``gcs`` = Google Cloud Storage; ``gdrive`` = a locally
MOUNTED
    Google Drive (``gdrive.mount_root``). Per-backend settings are loose dicts passed to the
constructor ?
    **secrets are never stored here**, only endpoints / regions / file paths / source-selectors
    (boto3/google auth chains supply credentials). See ``backend/storage/``."""
    backend: Literal["local", "s3", "gcs", "gdrive"] = "local"
    # Output root for produced artifacts. "" => fall back to output.output_dir (non-breaking).
    output_root: str = ""
```

7. assistant (2026-06-21T19:13:07.454Z)

Now let me confirm the `SecurityConfig` is actually wired into the top-level config under the key `security`.

8. user (2026-06-21T19:13:08.980Z)

```
origin/master:backend/config/models.py:280:class SecurityConfig(BaseModel):
origin/master:backend/config/models.py:404:    security: SecurityConfig =
Field(default_factory=SecurityConfig)
```

9. assistant (2026-06-21T19:13:11.294Z)

The field is named `security` on the root config (line 404). Let me confirm the enclosing class at line 404 is the root config model.

10. user (2026-06-21T19:13:12.228Z)

```
    enabled: bool = False
    pricebook_version: str = "placeholder-v0"   # the seeded $0 version; owner flips to a real
one
    fx_inr_per_usd: float = 83.0                  # USD?INR display rate (Bangalore market)
    margin: float = 0.0                            # resale markup (?0); 0 = bill at cost


class AppConfig(BaseModel):
    """Root configuration object.

    All runtime configuration should be accessed via an instance of this class
    (or the loader) rather than raw dict lookups.
    """

    sport: Sport = "badminton"
    ai_provider: str = "gemini"
    # Auto-tracking alias: 2.5-flash is legacy now that 3.5-flash is the stable flagship;
    # the alias never deprecates underneath us. Eval reports record the run date alongside.
    ai_model: str = "gemini-flash-latest"

    validation: ValidationConfig = Field(default_factory=ValidationConfig)
    indexing: IndexingConfig = Field(default_factory=IndexingConfig)
    output: OutputConfig = Field(default_factory=OutputConfig)
    stitching: StitchingConfig = Field(default_factory=StitchingConfig)
```

```
    security: SecurityConfig = Field(default_factory=SecurityConfig)
    logging: LoggingConfig = Field(default_factory=LoggingConfig)
    storage: StorageConfig = Field(default_factory=StorageConfig)
    deployment: DeploymentConfig = Field(default_factory=DeploymentConfig)
    billing: BillingConfig = Field(default_factory=BillingConfig)

    # Allow extra keys during transition so existing config.json files do not break.
```

11. assistant (2026-06-21T19:13:19.089Z)

All confirmed. Let me cross-check the doc's described meanings against the code:

- `edge_auth_enabled: bool = False` ? doc says "turn the CF-Access JWT verify ON"; default False. Doc §7 says "You MUST set `edge_auth_enabled=true` to expose." Matches ? default is OFF (False), not true.
- `allowed_video_root: Optional[str] = None` ? doc says "the path-jail ? REQUIRED when exposed". Code comment: "/api/process video_path must resolve under this directory". Matches.
- `upload_dir: str = "output/uploads"` ? doc says "MUST sit UNDER allowed_video_root". Code comment confirms same constraint. Matches.
- `cf_team_domain: str = ""` ? doc: "https://<your-team>.cloudflareaccess.com". Code comment matches (issuer + JWKS). Matches.
- `cf_access_aud: str = ""` ? doc: "the CF Access application AUD". Code comment matches. Matches.

All 5 keys exist under a `security` section, types and defaults align, and the doc's `edge_auth_enabled` default is correctly OFF (False), not true. The doc's described meanings match the code comments precisely.

12. user (2026-06-21T19:13:31.192Z)

Structured output provided successfully

13. assistant (2026-06-21T19:13:37.465Z)

Verification complete. The claim is **confirmed**.

All 5 keys named in DEPLOY_ALPHA.md §2.2 exist in the engine's `SecurityConfig` on origin/master, attached to the root config under the `security` key, with types, defaults, and documented meanings all matching the doc:

- `allowed_video_root: Optional[str] = None` ? path-jail (models.py:283)
- `upload_dir: str = "output/uploads"` ? must sit under allowed_video_root (models.py:289)
- `edge_auth_enabled: bool = False` ? DEFAULT-OFF, correctly not `true` (models.py:303)
- `cf_team_domain: str = ""` (models.py:304)
- `cf_access_aud: str = ""` (models.py:305)

The section is named `security` (models.py:404), and the boot posture in startup.py reads exactly these keys. No discrepancy. One non-blocking note for the owner: SecurityConfig carries additional keys (`max_upload_mb=4096`, `max_part_mb=95`, `allowed_upload_extensions`) the doc doesn't name ? all with safe defaults, so no action needed.